

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1003	Course Title:	Software Engineering
CO Assessed:	CO1 AT1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	I.Guru Hemanth Reddy	Reg. No.:	192311222
Date:	22/07/26	Duration:	60 minutes

Code of Conduct

I Guru Hemanth Reddy(19231122) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _Hemanth Reddy

Annexure A – Case Study Analysis

Smart College Bus Tracking and Management System

1.Understanding the Problem Statement

1.1 Introduction and Problem Context

Transportation is a critical logistical component in the daily operations of educational institutions. Many colleges maintain an extensive fleet of buses to transport students and faculty across various routes spanning urban and suburban areas. However, as the student population grows and traffic dynamics become more complex, the reliance on traditional, static transportation management systems has proven highly inadequate. The absence of a technological backbone in college transport leads to inefficiencies, frustration, and compromised safety, necessitating the development of a Smart College Bus Tracking and Management System.

1.2 Detailed Problem Explanation

The core problem stems from the unpredictability and opacity of the current transit operations. Under the existing framework, students rely on static paper schedules that do not account for real-time variables such as traffic congestion, vehicular breakdowns, or adverse weather conditions. Consequently, students often experience extended wait times at bus stops, leading to missed classes and academic disruption. Conversely, buses may depart early, leaving students stranded.

From the administration's perspective, the transport department operates blindly once the bus leaves the depot. There is no mechanism to monitor driver behavior, track route deviations, or dynamically reroute buses in response to emergencies. Communication is heavily fragmented; notifying students of a delayed or canceled bus requires manual, asynchronous interventions (e.g., notice boards or delayed email blasts), which are rarely seen in time.

1.3 Objectives of the Proposed System

To address the aforementioned issues, the proposed Smart College Bus Tracking and Management System is designed with the following primary objectives:

- **Real-Time Visibility:** To provide students, parents, and administrators with the exact live GPS location of all operational college buses on an interactive map.
- **Accurate ETA Prediction:** To calculate and display highly accurate Estimated Times of Arrival (ETA) based on live traffic data and vehicle speed.
- **Automated Communication:** To establish an automated push-notification system that alerts users to delays, route changes, or proximity alerts when a bus is approaching their designated stop.
- **Administrative Control:** To empower the college transport department with a centralized dashboard for managing routes, drivers, and fleet maintenance schedules.
- **Data-Driven Analytics:** To collect historical transit data to identify bottlenecks, optimize routes, and reduce fuel consumption.

1.4 Scope of the Project

The scope of the project defines the boundaries of the system.

- **In-Scope:**
 - Development of a cross-platform mobile application (iOS/Android) for students and faculty.
 - Development of a specialized mobile application for bus drivers to broadcast GPS data and signal emergencies.
 - Development of a secure web-based administrative dashboard for transport managers.
 - Integration with third-party Mapping and Routing APIs (e.g., Google Maps API or Mapbox).
 - Cloud-based backend infrastructure capable of handling high-frequency telemetry data.

- **Out-of-Scope:**
 - Payment gateway integration (as the college transit is assumed to be pre-paid via tuition).
 - In-bus hardware installation beyond providing drivers with GPS-enabled mobile devices.
 - Public transit tracking (the system is exclusively for private college fleets).

2. Stakeholders and Challenges

2.1 Identification of Stakeholders

A stakeholder is any individual, group, or entity that affects or is affected by the system. The primary stakeholders for this project are: [1] **Students and Faculty (End-Users)** [2] **Bus Drivers** [3] **Transport Administrators (College Management)** [4] **Parents / Guardians** [5] **System Maintainers (IT Support Staff)**

2.2 Roles, Responsibilities, and Challenges

Stakeholder	Responsibilities	Challenges Faced in Current System
Students & Faculty	Wait at designated stops at prescribed times; display valid ID for boarding.	Prolonged waiting times in harsh weather; anxiety over missed classes; lack of information regarding breakdowns; inability to plan transit efficiently.
Bus Drivers	Safely navigate assigned routes; adhere strictly to schedules; report maintenance issues manually.	Unaware of the number of waiting passengers; difficult to communicate emergencies to the college while driving; blamed for delays beyond their control (e.g., traffic).
Transport Admins	Schedule routes; assign drivers; manage vehicle maintenance; respond to student/parent grievances.	Inability to track fleet in real-time; manual and error-prone scheduling; difficulty verifying driver adherence to routes; high administrative overhead responding to ETA queries.
Parents / Guardians	Ensure their wards reach the college safely (particularly for late-night classes).	Anxiety regarding the safety and whereabouts of their children; lack of transparency in the event of college transit accidents or severe delays.
IT Support Staff	N/A (Role begins upon deployment).	In the existing manual system, IT is disconnected from transit, resulting in missed opportunities for automation and data utilization.

3.Current System Analysis

3.1 Overview of Existing Process

The existing transit process is predominantly manual. At the beginning of the semester, the transport department drafts a static schedule outlining bus numbers, routes, and expected arrival times at various stops. These schedules are printed and distributed, or uploaded as static PDF files to the college website. When a driver begins their shift, they take physical possession of the bus keys and a paper logbook. Communication between the driver and the transport office occurs exclusively via personal cellular phone calls.

3.2 Strengths of the Current System

While archaic, the current system possesses a few baseline strengths: - **Zero Technical Dependency:** The system does not rely on software, meaning it cannot “crash” due to server downtime or software bugs. - **Low Operational Cost:** There are no cloud hosting fees, software licensing costs, or IT personnel required to manage the transit process. - **Simplicity:** The manual process is universally understood; no training is required to read a printed schedule.

3.3 Weaknesses of the Current System

The weaknesses heavily outweigh the strengths in a modern educational environment: - **Static Inflexibility:** Schedules cannot dynamically adapt to real-world conditions. A 10-minute traffic delay cascades through the entire route without any mechanism to warn downstream passengers. - **Information Silos:** When a bus breaks down, only the driver knows immediately. It requires a manual phone call to the admin, who must then manually figure out how to notify students (often failing to do so in time). - **Poor Resource Allocation:** Some buses may run entirely empty on certain days, while others are dangerously overcrowded. Without data, the administration cannot reallocate vehicles efficiently.

3.4 Operational Problems and Systemic Gaps

The most significant gap is the “**Communication Void.**” Between the time a bus leaves the depot and the time it returns, it exists in a black box. The administration cannot verify if a driver skipped a stop, drove recklessly, or deviated from the approved route. Furthermore, there is a **Security Gap**; in the event of an accident or emergency, emergency responders cannot be provided with immediate, accurate GPS coordinates of the college bus, delaying critical assistance.

4. Proposed Software Solution

4.1 Overall System Architecture

The proposed solution utilizes a modern **Cloud-Based Client-Server Architecture** utilizing microservices. It consists of three primary client interfaces (Student App, Driver App, Admin Web Portal) communicating with a centralized cloud backend via RESTful APIs and WebSockets (for real-time telemetry). The backend utilizes a high-throughput messaging queue to process continuous GPS coordinates and persist them in a cloud database.

4.2 Functional Modules

The system is logically partitioned into the following core modules: [1] **User Authentication & Authorization Module:** Handles secure login via OAuth 2.0, utilizing college Single Sign-On (SSO) credentials. Differentiates access rights between students, drivers, and admins. [2] **Real-Time GPS Tracking Module:** The core engine. Receives high-frequency GPS coordinate streams from the Driver App, processes them against geospatial mapping APIs, and broadcasts live positions to the Student App. [3] **Route Management & Geofencing Module:** Allows admins to draw routes and define stops on a digital map. Geofencing triggers automated events (e.g., sending an alert when a bus enters a 1-kilometer radius of a stop). [4] **Notification Engine:** A push notification and SMS gateway responsible for dispatching proximity alerts, emergency broadcasts, and schedule changes. [5] **Administrative**

Dashboard & Analytics Module: Visualizes live fleet status, generates attendance/utilization reports, and flags anomalies (e.g., overspeeding).

4.3 Functional Requirements

- **FR1:** The system shall allow students to view the real-time location of buses on a digital map.
- **FR2:** The system shall calculate and display the ETA for the bus at the student's selected stop.
- **FR3:** The system shall allow users to set customizable proximity alerts (e.g., "Notify me when the bus is 5 minutes away").
- **FR4:** The Driver App shall continuously transmit GPS coordinates at an interval of no more than 5 seconds while a trip is active.
- **FR5:** The Driver App must include an easily accessible "Panic/Emergency" button that instantly alerts the administration.
- **FR6:** The Admin portal shall allow authorized personnel to add, modify, or delete buses, drivers, and routes.
- **FR7:** The system must generate weekly graphical reports on route utilization and average delays.

4.4 Non-Functional Requirements

- **Performance & Latency:** Location updates on the student app must reflect the driver's actual position with a latency of less than 3 seconds.
- **Scalability:** The backend must concurrently support up to 10,000 active student connections during peak morning and evening transit hours.
- **Availability:** The system requires 99.9% uptime during operational college hours (e.g., 6:00 AM to 10:00 PM).
- **Security:** All data in transit must be encrypted using TLS 1.3. User passwords must be hashed using bcrypt or Argon2.
- **Usability:** The mobile application must have an intuitive interface, requiring no more than 3 screen taps for a student to find their bus ETA.

4.5 Software Engineering Principles

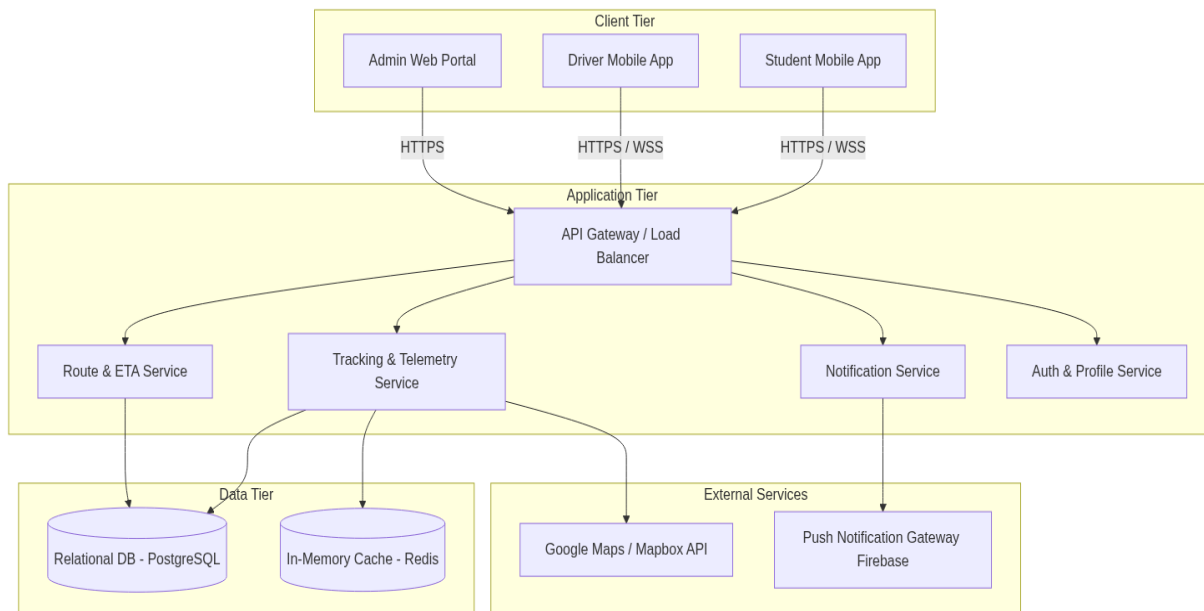
The architecture strictly adheres to core software engineering principles to ensure longevity and quality:

- **Modularity:** By dividing the system into distinct modules (Authentication, Tracking, Notifications), development teams can work concurrently. Changes in the notification engine will not break the GPS

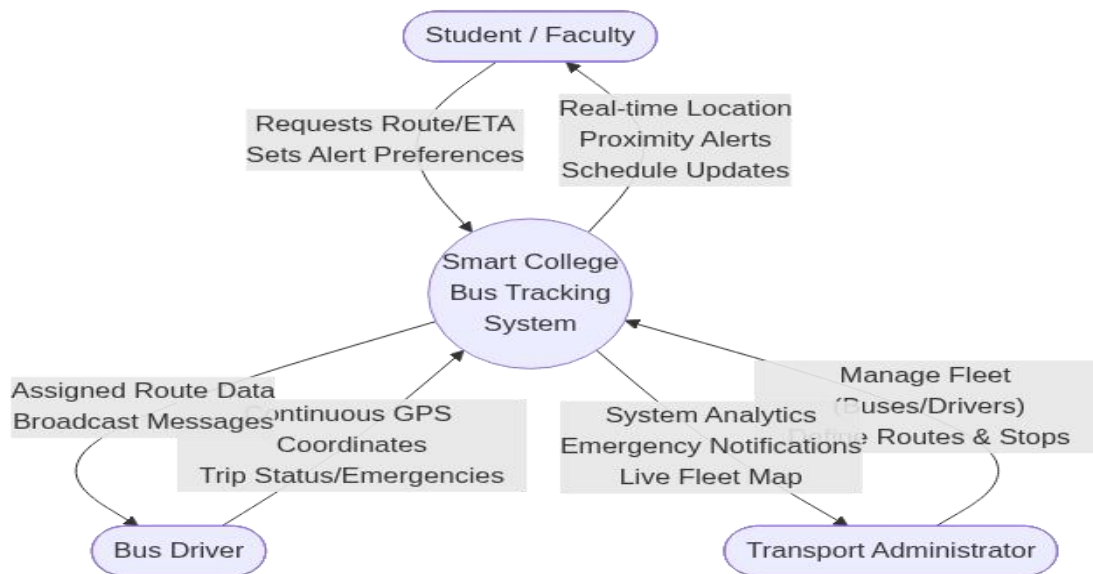
tracking logic. - **Scalability:** Utilizing cloud infrastructure (like AWS or Azure) and containerization (Docker/Kubernetes) ensures that server resources can auto-scale dynamically during peak morning hours. - **Maintainability:** The codebase will follow strict coding standards (e.g., SOLID principles) and utilize comprehensive documentation, making it easy for future developers to update the system. - **Reliability:** Automated failovers and database replication guarantee that the tracking system will continue to function even if a primary server node experiences a hardware failure. - **Security:** Adherence to principle of least privilege ensures drivers cannot alter routes and students cannot view administrative panels. - **Usability:** User-Centered Design (UCD) focuses on high-contrast maps, large typography, and simple iconography for rapid comprehension, even while walking.

4.6 System Modeling (Mermaid Diagrams)

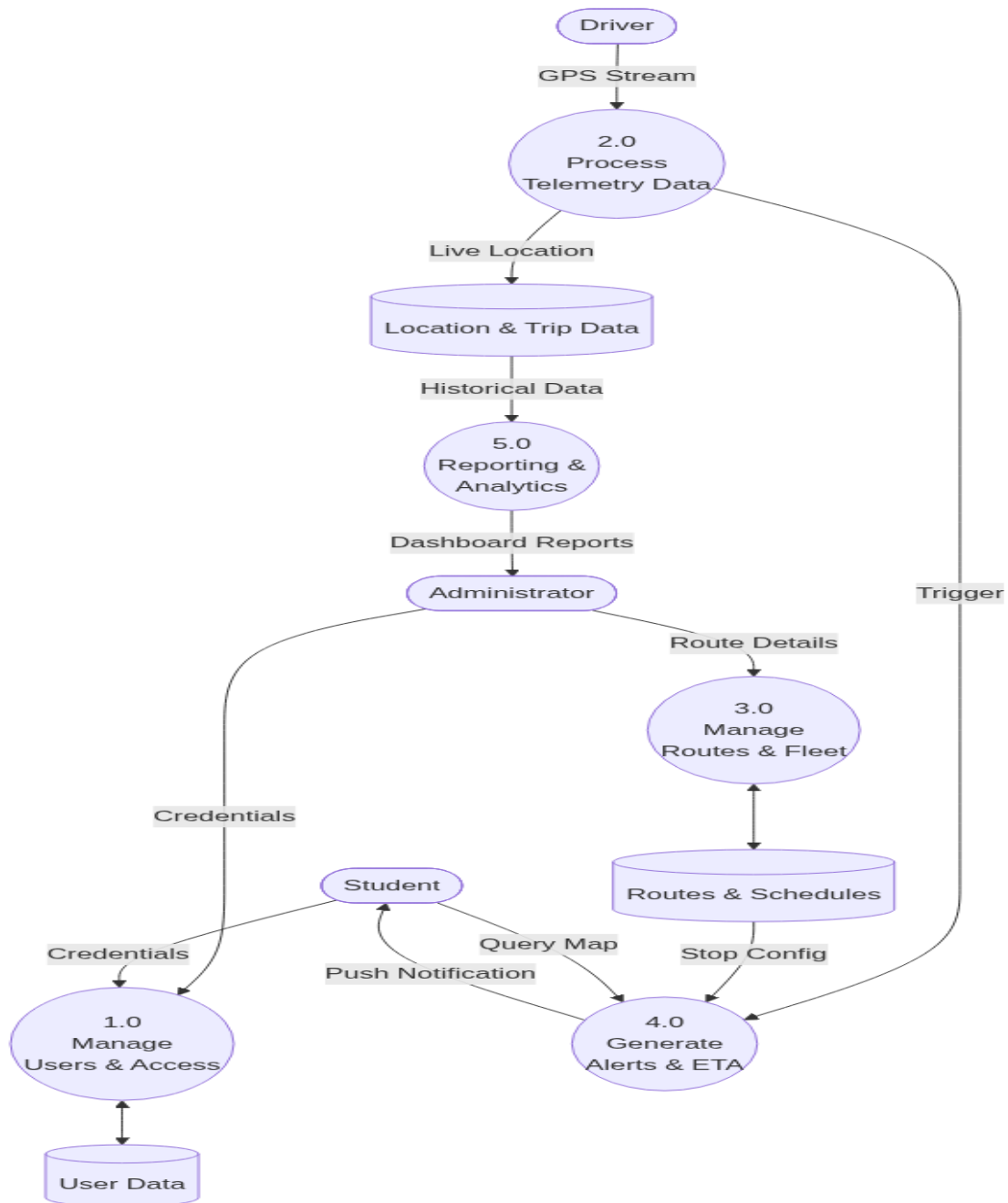
4.6.1 System Architecture Diagram



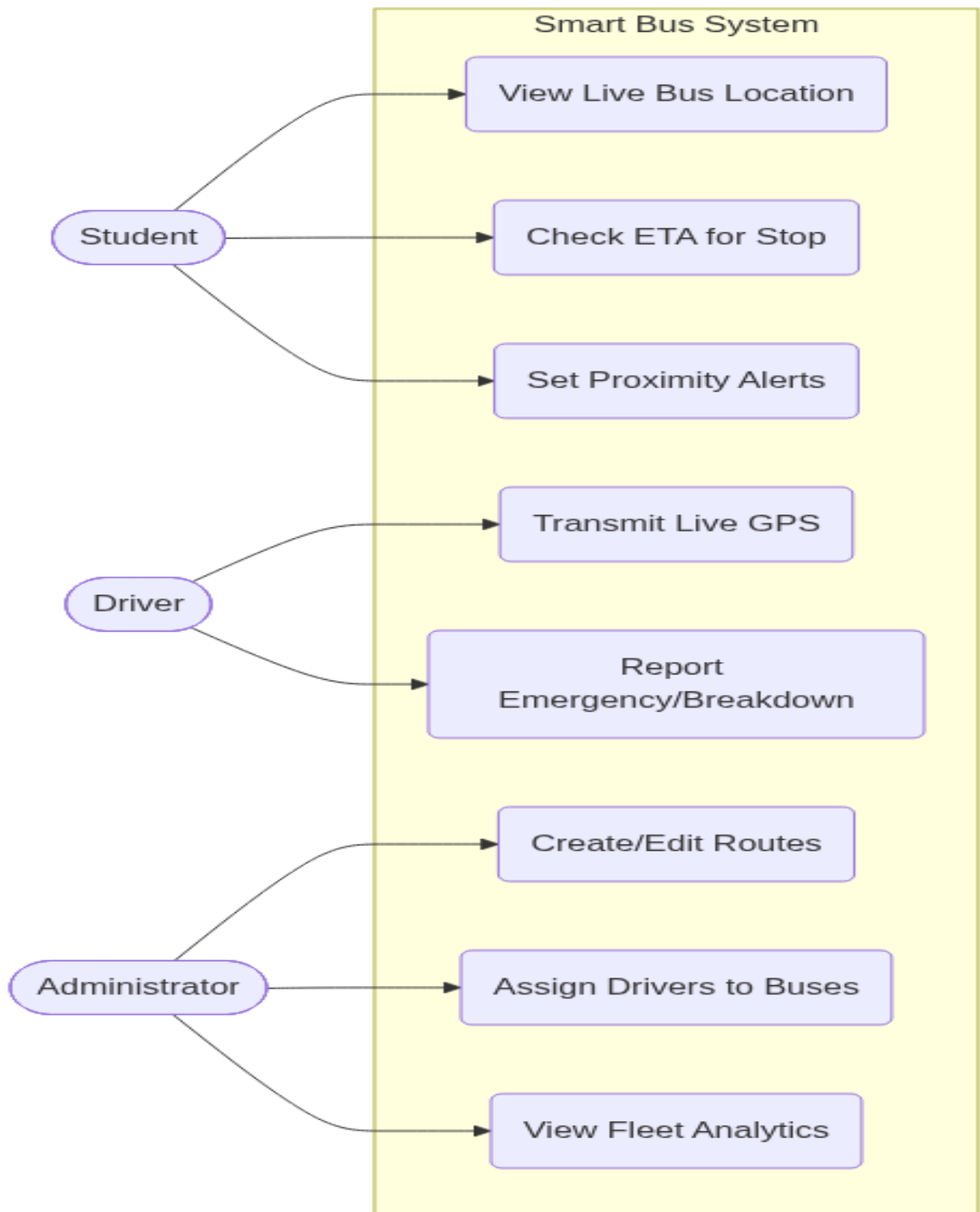
4.6.2 DFD Level 0 (Context Diagram)



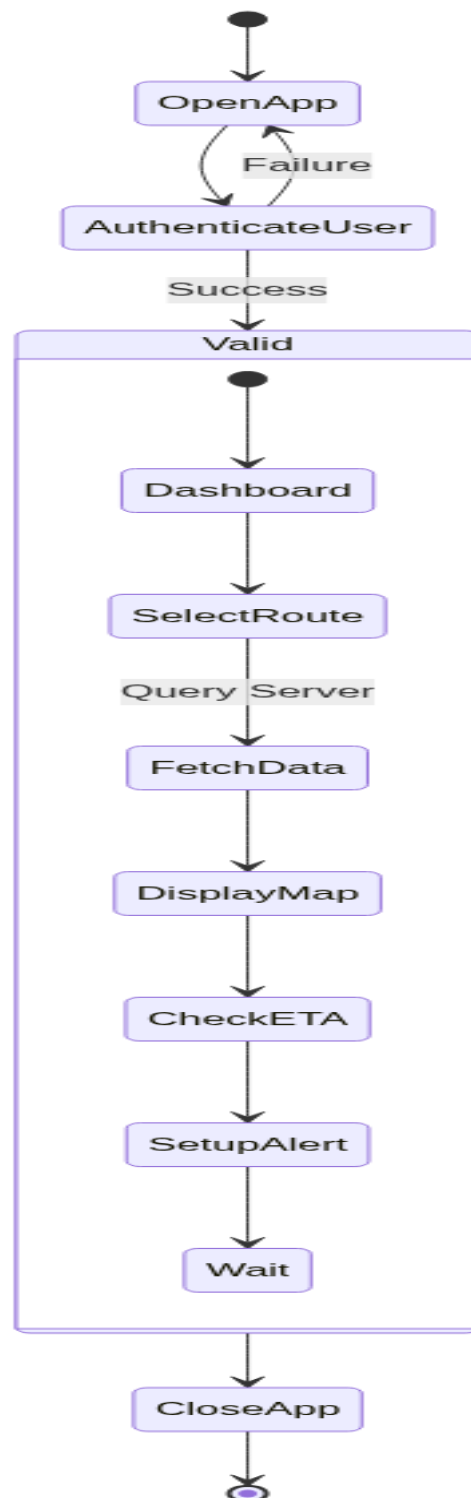
4.6.3 DFD Level 1



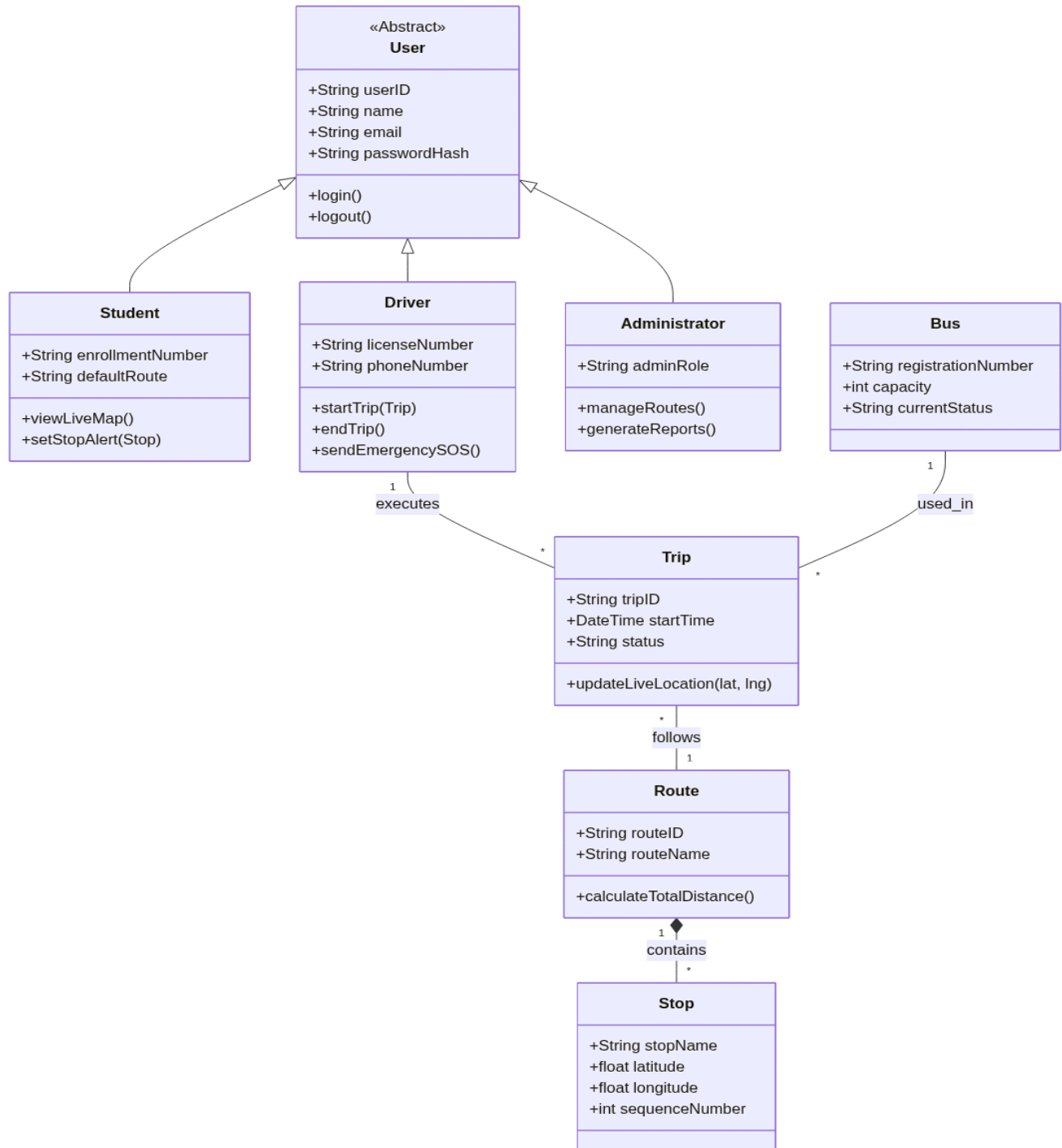
4.6.4 Use Case Diagram



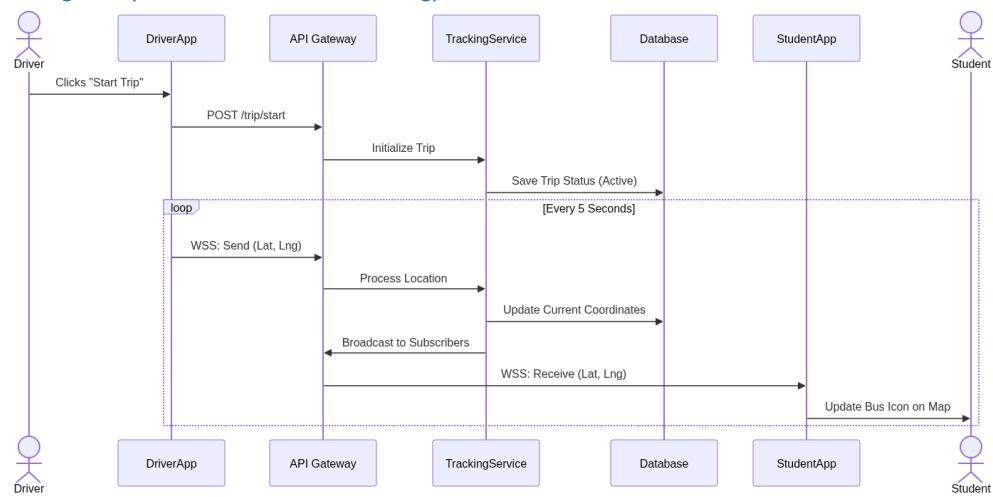
4.6.5 Activity Diagram (Student Checking Bus Location)



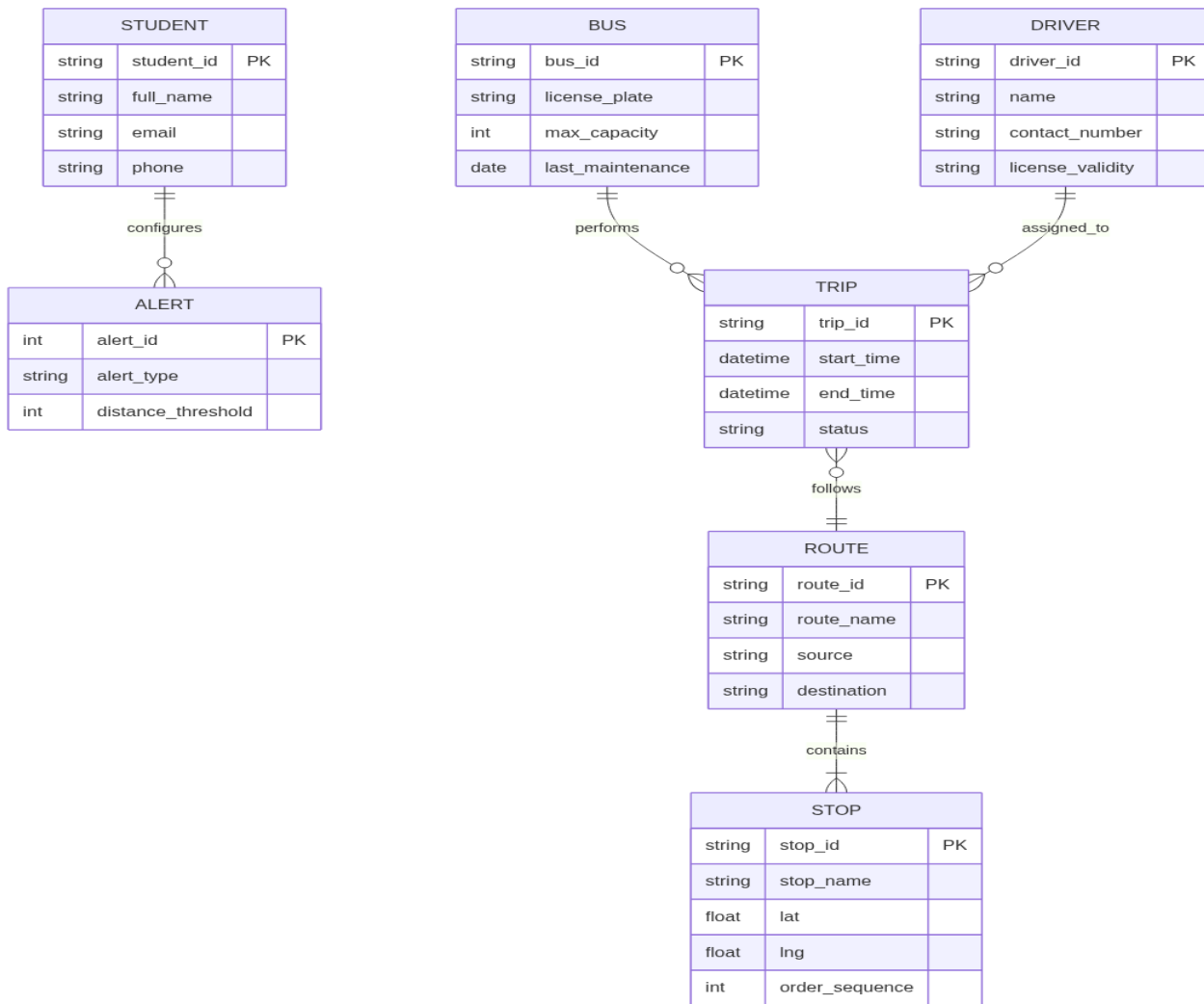
4.6.6 Class Diagram



4.6.7 Sequence Diagram (Real-Time GPS Tracking)



4.6.8 Entity Relationship (ER) Diagram



5. SDLC / Methodology

5.1 Selection of Agile Scrum

For the development of the Smart College Bus Tracking System, the **Agile Scrum Methodology** is selected as the most appropriate Software Development Life Cycle (SDLC) model. Scrum is an iterative and incremental framework designed to deliver high-value software rapidly while adapting to evolving requirements.

5.2 Why Agile Scrum is Suitable

Unlike the rigid Waterfall model, Agile Scrum is perfectly suited for this project because: - **Evolving Requirements:** The college administration may discover new needs (e.g., specific reporting metrics) only after seeing the initial prototype. Agile accommodates requirement churn gracefully. - **Faster Time-to-**

Market (Students Need It Now): The college cannot wait two years for a complete system. Agile allows for a Minimum Viable Product (MVP)—such as basic GPS tracking without advanced analytics—to be released to students within a few months. - **Continuous Feedback Loop:** By delivering working software in short cycles, developers can test the mobile app with real drivers and students, identifying UX/UI issues (like the driver app draining battery too fast) early in the process.

5.3 Scrum Artifacts and Roles

- **Product Owner:** A representative from the College Transport Department who defines the vision and prioritizes the Product Backlog.
- **Scrum Master:** Facilitates the process, removes technical blockers (e.g., getting API keys for Google Maps), and ensures Agile principles are followed.
- **Development Team:** A cross-functional team of UI/UX designers, backend engineers, and mobile developers.
- **Product Backlog:** A prioritized list of all desired features (Epics and User Stories). For example:
“As a student, I want to receive a push notification 5 minutes before the bus arrives, so I don’t wait in the rain.”

5.4 Sprint Ceremonies

The project will be executed in 2-week iterations known as Sprints. - **Sprint Planning:** At the start of the two weeks, the team selects the highest priority stories from the Product Backlog to form the Sprint Backlog. For Sprint 1, this might be building user login and basic map rendering. - **Daily Scrum (Stand-up):** A 15-minute daily meeting where developers answer: What did I do yesterday? What will I do today? Are there any impediments? - **Sprint Review:** At the end of the 2 weeks, the team demonstrates the working software (e.g., a working driver app sending coordinates to a local server) to the Product Owner and stakeholders for validation. - **Sprint Retrospective:** The team analyzes their internal processes to identify what went well and what requires improvement for the next sprint (e.g., deciding to shift from REST to WebSockets for faster location updates).

5.5 Incremental Development Strategy

The system will be built incrementally: - **Increment 1 (MVP):** Driver app broadcasting location; Student app showing location on a map. No ETA calculations, no alerts. - **Increment 2:** Implementation of the routing engine and ETA calculation algorithms. - **Increment 3:** Addition of the Push Notification Engine and proximity alerts. - **Increment 4:** Administrative dashboard, historical analytics, and reporting tools.

6. Expected Outcomes

6.1 Anticipated Benefits

- **Vastly Improved Student Experience:** The elimination of uncertainty regarding bus arrivals reduces student stress, waiting times, and tardiness to classes.
- **Enhanced Safety and Security:** Parents and college authorities have peace of mind knowing the exact location of buses. In emergencies, exact coordinates can be instantly relayed to authorities.

- **Data-Driven Operational Efficiency:** The administration can analyze historical data to identify inefficient routes, underutilized buses, or drivers who frequently deviate from protocols, leading to significant fuel savings.

6.2 Challenges Solved

- **The Communication Gap:** Solved via automated push notifications and instant admin alerts.
- **Static Scheduling Issues:** Solved by providing dynamic, real-time ETAs that factor in live traffic, replacing theoretical paper schedules.
- **Lack of Accountability:** Solved by persistent tracking, ensuring drivers adhere to assigned routes and speed limits.

6.3 Risks and Mitigation Strategies

Risk Factor	Impact	Mitigation Strategy
Mobile Network Dead Zones	Loss of GPS data transmission.	The Driver App will cache GPS coordinates locally and burst-upload them once network connectivity is restored. Extrapolative algorithms will estimate position during brief outages.
Driver Smartphone Battery Drain	App shuts down mid-trip.	Optimize the app to run efficiently in the background. Provide dedicated chargers or college-issued devices hardwired into the bus dashboard.
Low User Adoption	System fails due to lack of use.	Mandate system use for drivers. Run an awareness campaign for students highlighting the convenience factor.

6.4 Limitations

- **Dependency on Third-Party APIs:** The system’s ETA accuracy is heavily dependent on the accuracy of the chosen mapping provider (e.g., Google Maps traffic data).
- **Hardware Limitations:** The precision of the bus location is limited by the quality of the GPS sensor in the driver’s mobile device (usually accurate to within 5-10 meters, which may occasionally confuse nearby parallel roads).

6.5 Future Enhancements

Once the core system is stabilized, several advanced features can be integrated: - **AI-Driven Route Optimization:** Using machine learning to analyze months of traffic data to automatically suggest faster, more efficient route redesigns. - **RFID/NFC Student ID Integration:** Installing card readers on buses so students tap in/out, providing exact ridership metrics and ensuring no unauthorized boarding occurs. - **Electric Vehicle (EV) Fleet Integration:** Tracking bus battery charge levels and automatically scheduling routes that align with charging station availability.

7. Conclusion

The implementation of a Smart College Bus Tracking and Management System represents a vital digital transformation for the educational institution. By replacing opaque, manual processes with a transparent, real-time, cloud-based architecture, the college solves critical logistical challenges. The adherence to rigorous software engineering principles—specifically modular design and Agile Scrum methodology—ensures that the resulting software is robust, scalable, and highly adaptable to future needs. Ultimately, this system delivers an immediate, tangible improvement in safety, efficiency, and the overall quality of life for students, drivers, and administrators alike.

8. References

[1] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson. [2] Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education. [3] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org. [4] Fowler, M. (2014). *Microservices: A definition of this new architectural term*. MartinFowler.com. [5] IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. IEEE. [6] Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly Media.

End of Report